Designing aapsdkaspokd

Jonas Lux

May - October 2025

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Objectives . . . 1

2 Background and Fundamentals **1**
2.1 Modular Synthesizers and Eurorack . . . 1
2.2 Audio and CV fundamentals . . . 1

3 System Design **1**
3.1 System Requirements and Specification . . . 1
3.1.1 Functional Requirements . . . 1
3.1.2 Electrical Constraints . . . 1
3.1.3 Firmware Constraints . . . 1
3.2 System Architecture . . . 1
3.2.1 High Level System Overview . . . 1
3.3 Hardware Design . . . 3
3.3.1 Digital Hardware Design . . . 3
3.3.2 Analog Frontend Design . . . 3
3.3.3 Analog Input Stage Design . . . 3
3.3.4 Circuit Schematics . . . 4
3.3.5 ADC Absolute Input Limits: . . . 6
3.3.6 Noise Analysis . . . 6
3.3.7 PCB Design . . . 7
3.4 Firmware Architecture . . . 8
3.4.1 DMA, and Memory Layout . . . 8
3.4.2 RTOS Task Structure . . . 8
3.4.3 Drivers and Peripherals . . . 9
3.4.4 Numerical Architecture . . . 10
3.4.5 Calibration and Pitch Tracking . . . 11

4 Focus Point 1: Pitch Interpolation **13**
4.1 Problem Statement . . . 13
4.1.1 Decimation . . . 13
4.1.2 Upsampling . . . 13
4.1.3 Non-Integer Rate Conversion . . . 13
4.1.4 Naive Approach: ZOH and Linear Interpolation . . . 13
4.1.5 The Aliasing Problem . . . 14
4.2 Theory and Algorithm Selection . . . 14
4.3 Comparison of ZOH vs. Linear vs. Cubic Interpolation . . . 15
4.3.1 Time-Domain Comparison . . . 15
4.4 Frequency Response comparison . . . 18
4.5 Implementation . . . 19
4.6 Evaluation . . . 20
4.6.1 Measurement Setup . . . 20
4.6.2 Results . . . 21

5 Focus Point 2: Reverb **23**
5.1 Problem Statement . . . 23
5.2 Theory of the Schröder Reverb . . . 23
5.2.1 Feedback Comb Filter . . . 23
5.2.2 Allpass Filter . . . 28
5.3 Design and Implementation . . . 29
5.4 Evaluation . . . 32
5.4.1 Impulse Response Decay . . . 32
5.4.2 Spectrogram . . . 33

6 Verification and Evaluation **34**
6.1 Hardware Testing . . . 34
6.1.1 Power and safety tests . . . 34
6.1.2 Signal integrity measurements . . . 34
6.2 ADC Input Range and Signal Levels . . . 34
6.2.1 Noise and distortion measurements . . . 35
6.3 Firmware Testing . . . 35
6.3.1 Driver verification . . . 35
6.3.2 Timing and latency analysis . . . 35
6.3.3 Calibration accuracy . . . 35
6.4 System-Level Validation . . . 35
6.5 Summary . . . 35
6.5.1 Summary . . . 35
6.5.2 Ausblick . . . 35

References **36**

List of Figures

1 Hardware component diagram 2
2 System Software Flow Diagram 8
3 Interpolation method comparison 15
4 Interpolation method Frequency Response Comparison 18
5 FFT of 100Hz Saw signal - interpolated via Hermite and ZOH with different playback speeds 21
6 Schroeder Reverb Block Diagram 24
7 Feedback Comb Filter 25
8 Impulse and Frequency Response of Feedback Comb Filter with different feedback gain α 25
9 Filtered-Feedback Comb Filter with 1st order Lowpass 26
10 Impulse and Frequency Response of Lowpass Feedback Comb Filter with different feedback gain α and filter coefficient $d = 0.2$ 26
11 Allpass Filter 28
12 Impulse and Frequency Response of Allpass Filter 28
13 Schroeder Reverb Block Diagram 29
14 Reverb Decay Times of a 20 ms White Noise Burst 32
15 Reverb Tail Spectrogram at comb lowpass filter on feedback settings 33
16 Attenuation of a $10\,\mathrm{V_{pp}}$ Eurorack-level signal to a biased $1.32\,\mathrm{V_{pp}}$ ADC input. 34

1 Introduction

1.1 Motivation

Problem statement and Motivation

1.2 Objectives

Final Eurorack DSP capable hardware

2 Background and Fundamentals

2.1 Modular Synthesizers and Eurorack

2.2 Audio and CV fundamentals

3 System Design

3.1 System Requirements and Specification

3.1.1 Functional Requirements

3.1.2 Electrical Constraints

3.1.3 Firmware Constraints

3.2 System Architecture

3.2.1 High Level System Overview

../../images/componentdiagram_hardware.drawio.pdf

Figure 1: Hardware component diagram

3.3 Hardware Design

3.3.1 Digital Hardware Design

MCU selection MCUMCU

Crystal The NX2520SA-24.000000 MHz crystal was selected and verified for compatibility with the STM32H7 oscillator circuit following the procedure described in [3]. Crystal compatibility requires a gain margin of at least 5, defined as:

$$\text{Gain Margin} = \frac{g_{m,\text{crit,max}} \cdot 5}{g_{m,\text{crit}}} \geq 5$$

where the critical transconductance $g_{m,\text{crit}}$ is calculated from the crystal parameters as:

$$g_{m,\text{crit}} = 4 \cdot ESR \cdot (2\pi F)^2 \cdot (C_0 + C_L)^2$$

With $ESR = 100\ \Omega$, $F = 24$ MHz, $C_0 = 7$ pF, $C_L = 10$ pF, and $g_{m,\text{crit,max}} = 1.5$ mA/V for the STM32H7 [6, p. 107], the resulting gain margin is 7.31, confirming crystal compatibility.
The external load capacitors were calculated as:

$$C_{L1} = C_{L2} = 2 \cdot (C_L - C_s) = 2 \cdot (10\text{ pF} - 3\text{ pF}) = 14\,\text{pF}$$

where $C_s = 3$ pF accounts for MCU pin and PCB parasitic capacitance.

Audio Codec Selection

3.3.2 Analog Frontend Design

3.3.3 Analog Input Stage Design

Input AC Coupling The TLV320AIC series codec has a configurable input impedance of 10 kΩ, 20 kΩ, and 40 kΩ **TLV320AIC3204ApplicationReference2012**. To minimize loading on the source, the maximum available value of 40 kΩ is chosen.
The AC-coupling capacitor together with the input impedance of the codec ADC forms a simple RC high-pass filter, whose cutoff frequency can be calculated as

$$f_c = \frac{1}{2\pi R_{\text{eq}} C_c}$$

The resulting cutoff frequency f_c should be well below the audible range, typically under 20 Hz, to preserve bass integrity and prevent phase shifts that could degrade the perceived low-frequency response **audio˙phase˙shift**.
With a coupling capacitor of $C_c = 2.2\,\mu$F and $R_{\text{eq}} = 40\,\text{k}\Omega$, the cutoff frequency is

$$f_c = \frac{1}{2\pi \cdot 40\,\text{k}\Omega \cdot 2.2\,\mu\text{F}} \approx 1.81\,\text{Hz}$$

Input Scaling The TLV datasheet specifies a maximum input voltage on the analog pins of $0.5\,V_{\text{RMS}}$ with a common-mode voltage of 0.9 V **slaa557**. To map Eurorack audio levels into this range, the signal must be attenuated appropriately. Typical Eurorack signals reach ±5 V, i.e., $10\,\text{V}_{\text{pp}}$ **doepfer˙a110-2**.
The target ADC peak-to-peak voltage is calculated from the RMS specification:

$$V_{\text{ADC,peak}} = V_{\text{ADC,RMS}} \cdot \sqrt{2} \approx 0.707\,\text{V}$$

$$V_{\text{ADC,pp}} = 2 \cdot V_{\text{ADC,peak}} \approx 1.414\,\text{V}$$

The required attenuation factor is then

$$a = \frac{V_{\text{ADC,pp}}}{V_{\text{Eurorack}}} = \frac{1.414}{5} \approx 0.283$$

To satisfy the input impedance requirements while providing this attenuation, a **unity-gain buffer op-amp** is used before a passive resistor divider. The buffer presents an input impedance equal to the op-amp's own input impedance, which is typically greater than $200\,\mathrm{k}\Omega$, thereby easily meeting the desired $100\,\mathrm{k}\Omega$ input impedance at the module. The resistor divider can then be designed to provide the necessary attenuation without significantly loading the signal source.

Design Rationale The chosen signal-conditioning approach uses a unity-gain buffer followed by a passive resistor divider. This decision is based on the constraints of the system and the electrical design conclusions of other commonly used amplification circuits. An inverting amplifier topology is unsuitable because its input impedance is determined directly by the input resistor R_{in}. Since the module requires an input impedance on the order of $100\,\mathrm{k}\Omega$, the corresponding feedback resistor R_{f} would also need to be of similar magnitude in order to achieve the desired gain. Such large resistor values introduce significant thermal noise, as described by the Johnson noise expression

$$v_{\mathrm{n}} = \sqrt{4kTR},$$

which grows with increasing resistance and would negatively affect the signal-to-noise ratio. Furthermore, due to space and component constraints, adding a dedicated high-impedance buffer stage in front of the inverting configuration is not feasible.
A non-inverting amplifier configuration is also unsuitable because it cannot realize an attenuation. Its closed-loop gain is defined as

$$A_{\mathrm{v,noninv}} = 1 + \frac{R_{\mathrm{f}}}{R_{\mathrm{g}}},$$

and therefore satisfies

$$A_{\mathrm{v,noninv}} \geq 1,$$

which prohibits any gain smaller than unity. Since the ADC input range requires attenuation of the incoming signal, a non-inverting topology cannot be used.
For these reasons, the most practical and robust solution is to use a unity-gain buffer to ensure high input impedance and to isolate the source from the following attenuation network, and then apply a passive resistor divider to scale the signal to the required amplitude. The divider consists of $R_{\mathrm{top}} = 10\,\mathrm{k}\Omega$ and $R_{\mathrm{bot}} = 1.8\,\mathrm{k}\Omega$, resulting in a total resistance of $11.8\,\mathrm{k}\Omega$. The associated divider current is given by

$$I = \frac{V_{\mathrm{in}}}{R_{\mathrm{top}} + R_{\mathrm{bot}}}.$$

Thus, for an input voltage of $5\,\mathrm{V}$ the current is $I = 0.424\,\mathrm{mA}$, and for $12\,\mathrm{V}$ it increases to $I = 1.017\,\mathrm{mA}$. This power consumption is acceptable within the system's constraints, and the simplicity and predictability of the passive attenuation stage outweigh the modest inefficiency introduced by the resistor divider.

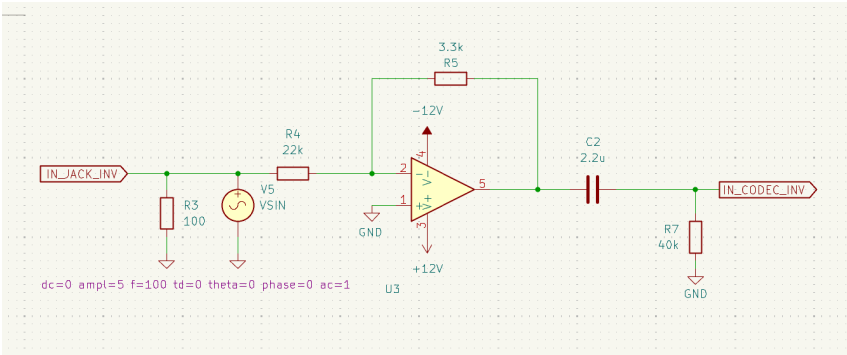
Comparison of Input Stage Circuits I ran SPICE simulations to compare two input stage approaches:

1. **Inverted Feedback Attenuator**
2. **Buffer to Voltage Divider**

3.3.4 Circuit Schematics

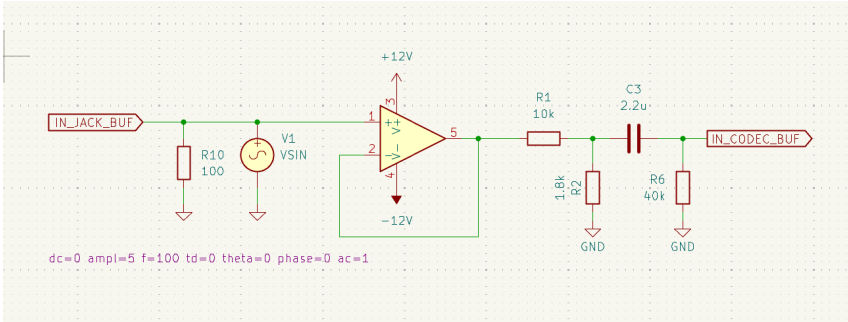
ADC input impedance is $40k\Omega$ and the signal source output impedance is 100Ω. The attenuation factor a is slightly different because of resistor values.

Inverted Feedback Attenuator

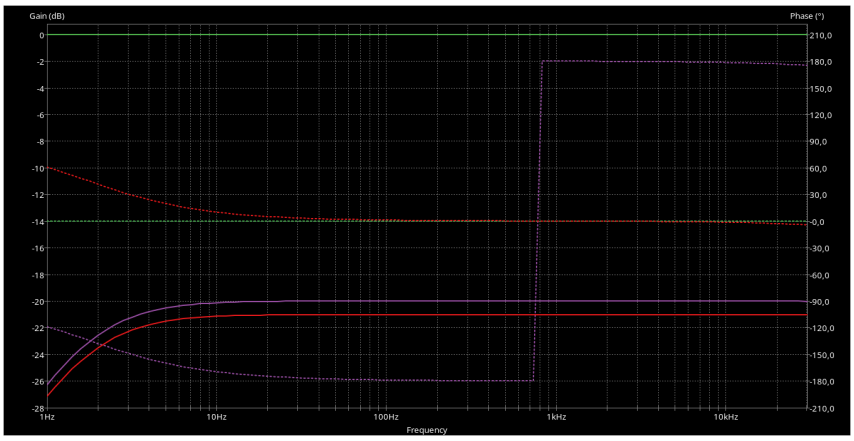


This circuit is used in most Mutable Instruments modules. It only offers $22k\Omega$ input impedance, and probably has higher thermal noise due to the resistors in the feedback path.

Buffer to Voltage Divider



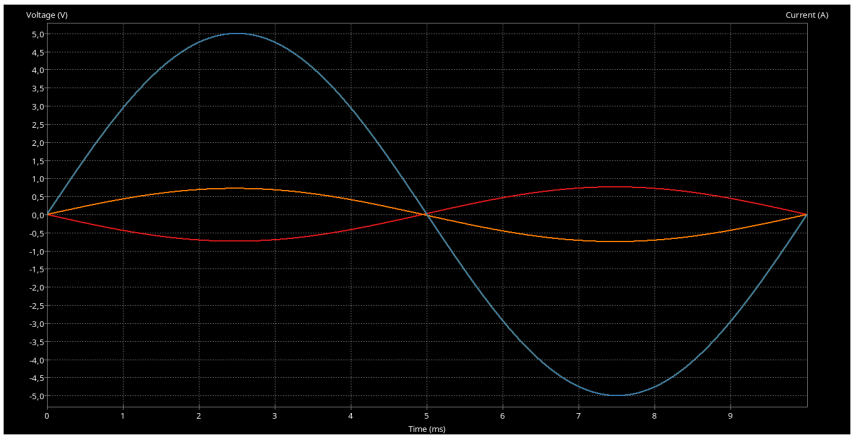
AC Analysis – Bode Plot The Bode plot shows the frequency response of both circuits in audio range ($1Hz - 30kHz$):



- **Green:** Input signal at $0dB$
- **Red:** Output of buffered circuit
- **Purple:** Output of inverted feedback circuit

Observation: Both circuits perform similarly, but the inverted feedback has a strong phase shift at 700–800Hz. The phase shift in the bass frequencies caused by the AC-coupling capacitor is below 10° at $20Hz$.

Transient Analysis – 100 Hz Signal



- **Blue:** Input signal ($10V_{pp}$, $100Hz$)
- **Red:** Output of inverted feedback circuit ($\sim 1.5V_{pp}$)
- **Orange/Red:** Output of buffered circuit ($\sim 1.47V_{pp}$)

3.3.5 ADC Absolute Input Limits:

The input pins of the ADC must remain within -0.3 V to ($A_{\text{VDD}} + 0.3$ V). Since $A_{\text{VDD}} = 1.8$ V, generated by the internal LDO of the TLV320AIC, the maximum allowed input voltage is 2.1 V.
To protect the ADC, a voltage divider with $R_1 = 1.8$ kΩ and $R_2 = 12$ kΩ is used, resulting in an attenuation factor of 0.15. For a maximum input signal of ±10 V, the attenuated voltage reaches approximately 2.11 V, which is just within the ADC limits. Such high input levels are uncommon in practice, as typical Eurorack audio signals are within ±5 V.

3.3.6 Noise Analysis

Inverted Input circuit In order to analyise the noie behaviour of an amplifier circuit, the respective noise components have to be calculated. In an inverted input op-amp circuit, the noise components contain of:

- Resistor Noise (Johnson Noise)
- op-amp voltage input noise
- op-amp current input noise

First the resistor noise aspects are analysed.
All resistors have a voltage noise given by:

$$V_{NR} = \sqrt{4kTBR}$$

where:

- T = absolute temperature in Kelvin, $T = T(°\text{C}) + 273.15$
- B = bandwidth in Hz
- k = Boltzmann's constant, $k = 1.38 \times 10^{-23}$ J/K
- R = resistance in Ohms

With values in the circuit: $R_{in} = 22\text{k}\Omega$ and $R_f = 3.3\text{k}\Omega$.
This yield following Johnson's noise densities:
For R_{in}

$$V_{Rin} = 155.846\text{nV}/\sqrt{Hz}$$

and for the feedback resistor R_f

$$V_{Rf} = 60.359\text{nV}/\sqrt{Hz}$$

at $T(°\text{C}) = 25°\text{C}$ and a bandwith of 20kHz

3.3.7 PCB Design

Layer Stackup

Power distribution

Signal integrity considerations

3.4 Firmware Architecture

3.4.1 DMA, and Memory Layout

3.4.2 RTOS Task Structure

The firmware employs FreeRTOS with three concurrent tasks organised by priority. Figure 2 illustrates the task structure and signalling flow:

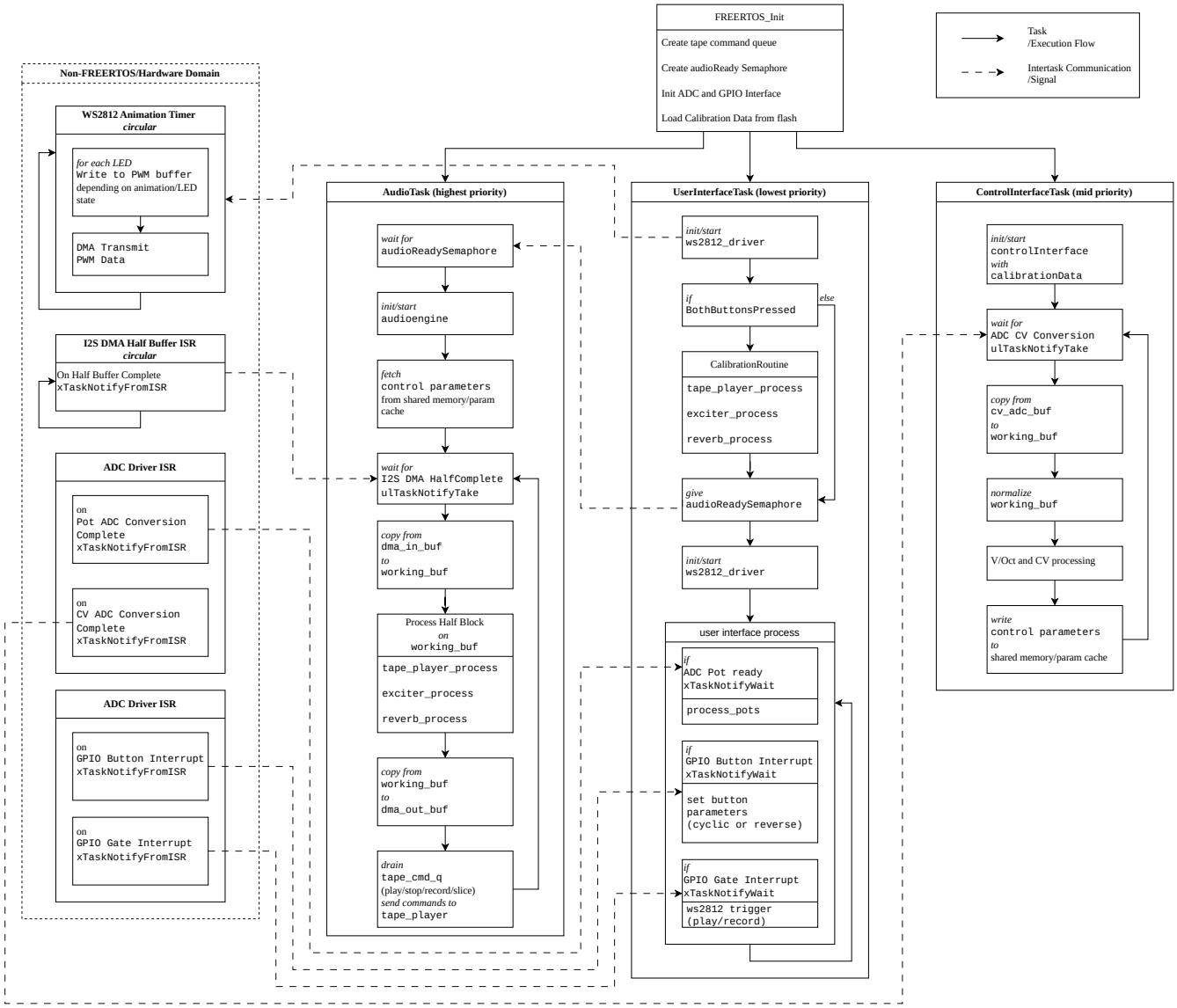


Figure 2: System Software Flow Diagram

AudioTask The highest-priority `AudioTask` is responsible for all real-time audio processing. After waiting on the binary semaphore `audioReadySemaphore`—which gates execution until the boot-time calibration routine has completed—it initialises the codec and audio engine, then enters a perpetual processing loop. Each iteration blocks on a direct-to-task notification issued by the DMA transfer-complete interrupt; upon wakeup, it fetches a consistent parameter snapshot via `param_cache_fetch`, processes one half-block of audio through the tape-player, exciter, and Schroeder reverb stages in sequence, writes the result back to the DMA output buffer, and drains any pending tape-transport commands from `tape_cmd_q` before yielding.

ControlInterfaceTask The mid-priority `ControlInterfaceTask` waits on task notifications from the ADC driver and, upon wakeup, calls `control_interface_process` to convert raw ADC readings into calibrated CV and pitch values.

UserInterfaceTask The lowest-priority `UserInterfaceTask` initialises the WS2812 LED driver and executes the boot-time calibration routine, during which the audio and control tasks are explicitly suspended via `vTaskSuspend` to give the calibration sequence exclusive CPU access. Once calibration completes, `audioReadySemaphore` is released and the task thereafter responds to pot-movement and GPIO notifications to update the LED state and parameter mappings.

Parameter Cache Parameter data propagated from the control and user-interface tasks to `AudioTask` bypasses FreeRTOS queuing entirely. `param_cache` implements a lock-free shared-memory scheme in which lower-priority tasks write individual fields using atomic store operations, while `AudioTask` reads a consistent snapshot each cycle via `param_cache_fetch`. This eliminates all blocking on the time-critical audio path and avoids priority inversion, following the established principle that real-time audio threads should never contend on a mutex **reinders2021**. Lock-free communication of this kind is well established in the real-time systems literature: Herlihy and Shavit provide a thorough theoretical grounding for wait-free and lock-free data structures **herlihy2012art**, while Butenhof gives a practical treatment of memory visibility and atomic operations in concurrent C programs **butenhof1997programming**.

Non-FreeRTOS Domain Outside the FreeRTOS task hierarchy, several interrupt-driven subsystems operate directly on the hardware. The I2S peripheral drives a circular DMA buffer that manages bidirectional audio data transfer between the codec and the MCU. A dedicated hardware timer controls the tick rate of the WS2812 animation system. GPIO interrupts notify `UserInterfaceTask` of button presses and gate-input events, and separately signal `ControlInterfaceTask` for gate-signal evaluation. The custom ADC driver raises ISRs on slide-potentiometer and V/Oct conversion-complete events and notifies the respective tasks when fresh data is available.

3.4.3 Drivers and Peripherals

Audio Codec Driver During hardware design, the versatile **TI TLV320AIC3204** stereo audio codec was chosen to handle analog-to-digital and digital-to-analog audio conversion.
The TLV320AIC3204 is a highly configurable audio codec controlled entirely through a register interface, exposing power management, input/output routing, gain stages, clocking, and pin multiplexing. It supports sample rates from 8 kHz to 192 kHz for both record and playback paths. The playback path provides programmable volume control, DSP filtering, and flexible DAC/analog mixing, while the record path supports both single-ended and differential input configurations with a digitally-controlled microphone preamplifier.
Clock generation is flexible: the internal clock can be derived from the `MCLK`, `BCLK`, or `GPIO` pins, or from an internal PLL whose input may likewise be sourced from any of these pins. The PLL accepts input clocks in the range of 512 kHz to 50 MHz, though its use is discouraged at the lowest power settings. The analog supply operates in the range of 1.5V–1.95V and the digital supply in the range of 1.26V–1.95V, with integrated LDOs accepting input voltages of 1.8V–3.6V. [8]
A common MCLK of 12.288MHz is generated by STM32H7's PLL tree **??**. This is then used to derivate all needed sychronization signals via its internal clock dividers.
The TLV320AIC3204 is configured over I^2C using a thin register-access layer built on the STM32 HAL. The two primitive operations, `codec_write_reg` and `codec_read_reg`, each perform a blocking HAL transfer to the codec's 7-bit address (`0x18`), with any HAL error escalated immediately to the system `Error_Handler`. Before initialisation, `codec_i2c_is_ok` probes the bus with `HAL_I2C_IsDeviceReady` to confirm the device is accessible.
The `codec_init` function executes a fixed register-write sequence to bring the codec into a known operating state. It begins with a software reset, then configures the clock dividers NDAC, MDAC, NADC, and MADC, whose values are selected at runtime from the `audio_sample_rate_t` argument according to the relation $f_s = f_{MCLK}/(N \cdot M \cdot \mathrm{OSR})$, with a 12.288 MHz master clock and a fixed OSR of 128. The I^2C interface is configured for 16-bit I^2S framing.

Subsequent writes on Page 1 enable the AVDD LDO, configure headphone soft-stepping, set the microphone PGA settling delay, and establish the reference charge time. The signal routing writes connect the DAC outputs to the line-output drivers LOL and LOR, and route the line inputs through the PGA to the ADC.
The sequence concludes on Page 0 by powering up the DAC and ADC blocks, setting unity gain, and unmuting all signal paths.

```
/* sample-rate-dependent dividers: fs = MCLK / (N * M * OSR) */
switch (rate) {
        case SAMPLERATE_48KHZ: n_val = 1; m_val = 2;  break;
        case SAMPLERATE_24KHZ: n_val = 1; m_val = 4;  break;
        case SAMPLERATE_8KHZ:  n_val = 1; m_val = 12; break;
        default:               n_val = 1; m_val = 2;  break;
}
codec_write_reg(0x0b, 0x80 | n_val); /* NDAC - MSB powers up divider */
codec_write_reg(0x0c, 0x80 | m_val); /* MDAC */
codec_write_reg(0x0e, 0x80);         /* DAC OSR = 128 */
```

Listing 1: TLV320AIC3204 clock divider configuration

Full register details and their respective configuration values are documented in TI's *TLV320AIC3204 Application Reference Guide* SLAA557, section 5.2, pp. 96–128 **TLV320AIC3204ApplicationReference2012**.

3.4.4 Numerical Architecture

The playback engine uses a mix of fixed-point and floating-point numbers. This design combines deterministic timing behaviour in the phase domain with numerically robust signal processing in the amplitude domain.

Fixed-Point Phase Accumulation Pitch-shifted sample playback is implemented using a fixed-point phase accumulator, as commonly employed in Direct Digital Synthesis (DDS) systems. The playback position is represented as an unsigned integer with fractional resolution (e.g., Q16.16 (`uint32_t`) or Q48.16 (`uint64_t`)format). The phase is updated according to

$$\phi[n+1] = \phi[n] + \Delta\phi, \tag{1}$$

where ϕ denotes the discrete phase and $\Delta\phi$ the phase increment corresponding to the desired playback frequency.
Using integer arithmetic for phase accumulation provides several advantages. First, wrap-around behaviour is clearly defined due to modulo-2^N overflow of unsigned integers. This eliminates the need for explicit boundary checks and ensures deterministic behaviour. Second, the frequency resolution is determined directly by the word length of the accumulator, which allows precise and stable pitch control over long playback durations. Floating-point accumulation, in contrast, may introduce small rounding errors that can accumulate over time and lead to phase drift in long-running systems.
For these reasons, fixed-point phase accumulators remain standard practice in oscillators, wavetable synthesizers, and embedded audio playback systems **lyons˙dsp**, **smith˙pasp**.

Floating-Point Signal Processing While the phase domain is handled using fixed-point arithmetic, all audio-domain processing is performed in single-precision floating-point. This includes mixing, biquad filtering, and reverberation.
The target platform is based on an ARM Cortex-M7 microcontroller with hardware support for single-precision floating-point operations. The processor provides a pipelined FPU capable of efficient multiply and accumulate operations, making floating-point digital signal processing computationally practical in real-time audio applications **arm˙m7˙trm**.
Floating-point arithmetic offers several important advantages for recursive and feedback-based structures such as IIR filters and reverberation networks. The wide dynamic range reduces the risk of overflow and

eliminates the need for manual scaling strategies that are typically required in fixed-point implementations. Furthermore, coefficient design tools and DSP literature predominantly assume floating-point arithmetic, simplifying implementation and validation **zoelzer˙dafx**.
In addition, floating-point processing improves numerical robustness in cascaded filter structures, where quantization effects in fixed-point arithmetic may otherwise degrade stability or frequency response accuracy.

Architectural Rationale The resulting architecture separates concerns between deterministic timing and numerical signal processing. Fixed-point arithmetic ensures stable and reproducible phase progression for accurate pitch control, while floating-point arithmetic simplifies implementation and improves robustness in the audio processing chain.
Such hybrid designs are widely used in modern embedded audio systems and digital synthesizers, particularly on microcontrollers equipped with hardware floating-point units. This approach achieves a balance between precision, performance, and implementation complexity appropriate for real-time embedded audio applications.

3.4.5 Calibration and Pitch Tracking

Calibration of V/Oct ADCs The chosen calibration routine implemented in this project is a simple 2-point calibration, where 2 known input values are converted by the ADC. From these values an equation of a straight line can be extracted, which will compensate external factors, like Opamp and electrical Ungenauigkeiten, as well as internal ADC offset and gain errors.
The implemented procedure is described in TI's "General ADC Calibration" article [7].
Two known input voltages X_0 and X_1 are applied to the ADC, producing digital codes S_0 and S_1.
Assuming a linear ADC model, the transfer function is

$$S = aX + b$$

where a is the gain (slope) and b is the offset (intercept at $X = 0$).
The calibration constants are computed as

$$a_c = \frac{S_1 - S_0}{X_1 - X_0}$$

$$b_c = S_0 - a_c X_0$$

During operation, the calibrated input value is obtained by

$$X = \frac{S - b_c}{a_c}$$

Translating the calibration to Pitch Tracking The general two-point ADC calibration can be directly mapped to the pitch tracking (V/Oct) system by interpreting the ADC conversion codes as normalized control voltages and the calibrated output value as pitch measured in semitones.
Two known reference voltages are applied to the V/Oct input:

- $X_0 = 1\,\mathrm{V}$, corresponding to musical note C1
- $X_1 = 3\,\mathrm{V}$, corresponding to musical note C3

The corresponding ADC conversion results are denoted as $c_{1,\mathrm{adc}}$ and $c_{3,\mathrm{adc}}$. These values are normalized to the range $[0, 1]$ using the ADC's max value of 2^16:

$$c_1 = \frac{c_{1,\mathrm{adc}}}{2^{16}}, \quad c_3 = \frac{c_{3,\mathrm{adc}}}{2^{16}}$$

Pitch is represented internally in semitones, with C1 defined as 12 semitones and C3 as 36 semitones. Assuming a linear relationship between normalized ADC value and pitch, the model becomes:

$$p = m \cdot c + b$$

where p is the pitch in semitones, c is the normalized ADC value, m is the pitch scaling factor, and b is the pitch offset.
The calibration constants are computed as:

$$m = \frac{36 - 12}{c_3 - c_1} = \frac{24}{c_3 - c_1}$$

$$b = 12 - m \cdot c_1$$

Pitch Tracking During operation, the pitch in semitones is calculated from a measured ADC value c as:

$$p = m \cdot c + b$$

Finally, the pitch is converted to a playback speed factor using the exponential relationship between pitch and frequency:

$$\text{playback_speed} = 2^{\frac{p}{12}}$$

4 Focus Point 1: Pitch Interpolation

4.1 Problem Statement

An obvious problem arises when dealing with variable speed playback of buffered audio:
the playback speed may not be an integer multiple of the original sample rate. This technique is generally known as **sample-rate conversion**. Its goal is essentially to obtain a new discrete representation $x_d[n]$ of the underlying continuous signal $x_c(t)$.

4.1.1 Decimation

The simplest case of sample-rate conversion is **decimation**: reducing the sample rate by an integer factor M. This is achieved by retaining only every M-th sample of the original sequence:

$$x_d[n] = x[nM] = x_c(nMT)$$

However, directly subsampling without prefiltering causes aliasing if the signal contains energy above the new Nyquist limit π/M. To avoid this, a discrete-time lowpass filter with cutoff π/M must be applied before compression. **oppenheimDiscretetimeSignalProcessing2014**
In the context of faster-than-unity playback, the playhead advances by more than one sample per output step, which is analogous to decimation — some input samples are skipped entirely.

4.1.2 Upsampling

The inverse operation is **upsampling**: increasing the sampling rate by an integer factor L. The classical approach inserts $L-1$ zeros between each pair of input samples to produce an expanded sequence:

$$x_e[n] = \begin{cases} x[n/L], & n = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise} \end{cases}$$

The zero-insertion creates spectral images of the original signal at multiples of $2\pi/L$. These are removed by a lowpass filter with cutoff π/L and gain L, which reconstructs the missing sample values. **oppenheimDiscretetimeSignalProcessing2014**

4.1.3 Non-Integer Rate Conversion

In practice, the playback speed of an audio sampler is not constrained to integer multiples of the original rate. A pitch shift of a perfect fifth corresponds to a factor of 1.5×; an arbitrary detuning may produce any irrational ratio. The classical integer-factor approach — upsample by L, filter, then downsample by M — can approximate any rational ratio L/M, but requires a different filter for each ratio and cannot smoothly vary over time **oppenheimDiscretetimeSignalProcessing2014**.
A more practical approach for real-time audio is to treat sample-rate conversion as a continuous interpolation problem: given the discrete samples $x[n]$, evaluate the signal at an arbitrary fractional position $t \in R$.
The playhead is represented as a fixed-point number with a fractional part, and the output at each step is computed by interpolating between the nearest input samples. The quality of this interpolation determines the spectral fidelity of the pitch-shifted output.

4.1.4 Naive Approach: ZOH and Linear Interpolation

The simplest possible strategy is **zero-order hold (ZOH)**: the output simply repeats the last known sample value until the playhead reaches the next integer position.

$$y_{\text{ZOH}}(t) = x_0, \quad t \in [0, 1)$$

ZOH requires no arithmetic at all, but produces a staircase-shaped waveform with discontinuities at every sample boundary, introducing significant high-frequency distortion. It is included here as a baseline to establish the minimum quality floor.

A straightforward improvement is **linear interpolation**: given two adjacent samples x_0 and x_1, the value at fractional offset $t \in [0, 1)$ is:

$$y_{\text{lin}}(t) = (1 - t)\, x_0 + t\, x_1$$

This requires only one multiply-accumulate per output sample, which makes it an obvious first candidate on constrained hardware. While linear interpolation eliminates the discontinuities of ZOH, it remains only C^0 continuous [1] – the interpolated values match at sample boundaries, but the slope changes abruptly at each one.

4.1.5 The Aliasing Problem

Despite its computational simplicity, linear interpolation has a fundamental spectral limitation: it is only C^0 continuous. The interpolated signal matches the original sample values at integer positions, but the slope (first derivative) changes abruptly at every sample boundary. Across an entire buffer, the output is a piecewise-linear waveform whose derivative is a train of rectangular pulses — a signal inherently rich in high-frequency harmonic content.

The aliasing problem is most severe during **upsampling**, i.e. at playback speeds below $1.0\times$, where the playhead advances by less than one sample per output step and the interpolator must synthesise the majority of output samples from scratch. At $0.5\times$ the interpolator is called for every other output sample; at $0.25\times$ it is called three times for every input sample consumed. The accumulated interpolation error produces aliasing artefacts that worsen monotonically as the playback speed decreases.

At playback speeds above unity, the playhead advances by more than one sample per output step and some input samples are skipped. The interpolator is still invoked for each output sample, but the fractional offset Δt between consecutive output samples varies less and the slope discontinuities at sample boundaries are smaller. Aliasing artefacts are therefore less prominent in the downsampling regime.

With linear interpolation, the piecewise-linear reconstruction introduces slope discontinuities at every sample boundary, producing harmonic artefacts between the signal's spectral lines that are clearly visible in an FFT of the output.

Therefore a higher-quality interpolation scheme is desirable, whose output is at least C^1 continuous, i.e. one where the slope is smooth across sample boundaries.

4.2 Theory and Algorithm Selection

A high quality resampling algorithm aims to maximise passband flatness and optimise the SNR (signal-to-noise ratio), i.e. not to alter the frequency content of the resulting signal. However, interpolation always introduces some degree of aliasing, resulting in degradation of the interpolated signal.

On an embedded platform, it is important to choose an algorithm that is mostly aliasing-free and computationally fast, while being aesthetically pleasing.

Interpolators can generally be classified as either IIR or FIR/polynomial. **juliuso.smithInterpolatedDelayLines2022** While FIR-based interpolation provides optimal band-limited reconstruction, its computational cost makes it impractical for real-time embedded audio systems. Polynomial Interpolators generally give decent results at low computational cost, which make them a solid choice for embedded realtime resampling. Polynomial interpolators calculate an underlying polynomial that passes through all the given control points in order to interpolate points. The greater the number of control points, the higher the degree of the polynomial.

A classic example of a polynomial interpolator is the *Lagrangian interpolation*, which finds an N order polynomial through $N + 1$ given control points. [5] When increasing the number of matching points with the Lagrangian polynomial, the interpolation converges with the sinc ideal interpolation. This level of accuracy can only be achieved with very high computational cost.

Another polynomial interpolation technique *cubic splines*, involves fitting a cubic (third-order) polynomial through two known points. It achieves smoothness by also adjusting the polynomials' slope, by changing its coefficients. [5] Hermite interpolation is part of the family of cubic *osculating interpolation*, in which

[1] A function is C^n continuous if its derivatives up to order n exist and are continuous everywhere. C^0 therefore denotes a function that is continuous but whose first derivative may be discontinuous. C^1 additionally requires a continuous first derivative. **weissteinSmoothFunction**

in addition to matching the function value at the control points, the points also match a number of derivatives at the control points. [4]
A good performance vs computation tradeoff is a 4-point Hermite interpolation.

4.3 Comparison of ZOH vs. Linear vs. Cubic Interpolation

4.3.1 Time-Domain Comparison

When comparing the naive linear approach with its more sophisticated counterpart graphically, it becomes clear that the additional computational overhead of cubic interpolation is justified.

In Figure 3, zero-order hold (1), linear interpolation (2), the Catmull-Rom cubic Hermite interpolation used in this project (3), and an ideal cubic Hermite interpolation with exact derivatives (4) are compared. The test signal is a sum of three sinusoids, simulating a complex audio signal:

$$x(t) = 0.8\sin(2t) + 0.6\sin(t) + 0.4\sin(2.7t) \tag{2}$$

../images/interpolation/fig_interpolation_comparison.pdf

Figure 3: Interpolation method comparison

The need for smooth interpolation becomes clear when the signal contains frequency content that approaches the sampling rate - the interpolator is forced to reconstruct steep, rapidly changing slopes from

very few samples. While this scenario would be prevented in practice by the Nyquist theorem, it serves as a useful stress test that exposes the characteristic artifacts of each method clearly.
Smooth interpolation is still directly relevant to the pitched playback feature of this project though. On the DSP level, pitch shifting is implemented as a fractional sample rate conversion: the playback pointer advances through the sample buffer at a non-integer step size, meaning most output samples fall between two existing input samples. The interpolator is therefore called on every single output sample, and its accuracy in estimating the true waveform shape between discrete points directly determines the fidelity of the pitched output. A smooth, accurate interpolator is essential to avoid waveform distortion artifacts in the reconstructed signal.

(1) Zero-Order Hold (ZOH) Zero-order hold is the simplest possible interpolation strategy: the output holds the last known sample value until the next sample is reached, producing a staircase-shaped waveform, it introduces no smoothing whatsoever.
Formally, the ZOH has an impulse response of unity over one sampling interval T_s, giving a frequency response of **moirRudimentsSignalProcessing2022**:

$$|H(j\omega)| = \left| T_s \operatorname{sinc}\left(\frac{\omega T_s}{2}\right) \right|$$

This sinc-shaped magnitude response causes the ZOH to attenuate high frequencies of the original signal, an effect known as *ZOH droop*. At the same time, the staircase discontinuities introduce new high-frequency harmonic content not present in the original signal. In the context of interpolation for pitch shifting, both effects contribute to audible distortion.

(2) Linear Interpolation Linear interpolation has no knowledge of the signal's history or trajectory — it connects each pair of samples with a straight line. This produces abrupt changes in slope at every sample boundary, which are especially visible near the peaks and troughs of the waveform, where the curvature of the underlying signal is greatest.

(3) Cubic Hermite with Exact Derivatives Since the test signal is analytically defined, its derivative is known exactly at every sample point:

$$x'(t) = 1.6\cos(2t) + 0.6\cos(t) + 1.08\cos(2.7t) \tag{3}$$

The resulting curve closely follows the original continuous signal throughout, including at the peaks and troughs where linear interpolation struggles most. This represents the theoretical upper bound of Hermite interpolation quality.

(4) Cubic Hermite with Catmull-Rom Tangents In a real-time audio context, the underlying signal and its derivatives are unknown. The Catmull-Rom approach addresses this by estimating the tangent at each point from its immediate neighbours. Despite relying on approximated rather than exact derivatives, the resulting curve closely resembles the ideal Hermite interpolation. The small deviation is practically negligible, making Catmull-Rom a well-justified tradeoff between computational simplicity and interpolation quality.
Another observation that can be made when looking at the graph, is that the first and last sample are not being interpolated. The Catmull-Rom approach needs the $n-1$ and $n+3$ neighbouring samples solely for the tangent estimation and is not able to interpolate from these.

4.4 Frequency Response comparison

../images/interpolation/fig_interpolation_freq_response.pdf

Figure 4: Interpolation method Frequency Response Comparison

When comparing frequency response of the 3 given interpolators, it becomes clear that Hermite has the least high frequency aliasing beyond of f_s. The continuity of the Hermite Interpolator – which is visible in the time domain (Section 4.3.1) – is directly reflected in the frequency domain. Smoother time-domain reconstruction corresponds to faster rolloff and lower aliasing lobes beyond Nyquist.
The Catmull-Rom / Hermite response exhibits a characteristic *punctured* sidelobe structure: each aliasing band is split into two sub-lobes by a notch, a consequence of the slight negative values in the kernel at $|t| \in [1, 2]$.
As noted by Niemitalo [4, p. 12], this puncturing reduces the peak aliasing energy significantly compared to Lagrange interpolators of the same order, while the passband remains nearly flat up to the Nyquist limit. This combination of flat passband and suppressed aliasing lobes makes the 4-point cubic Hermite a well-justified choice for real-time resampling on constrained hardware.

4.5 Implementation

Interpolation Function The algorithm is implemented as a single function that takes the phase accumulator and the sample buffer as parameters. As described in Section 3.4.4, this function is the boundary between fixed-point phase tracking and floating-point DSP computation: the integer part of the phase selec ts the sample index, while the fractional part is converted to a `float` to serve as the interpolation parameter $t \in [0, 1)$.

```
float hermite_interpolate(uint32_t phase, int16_t* buffer) {
        uint32_t idx = phase >> 16; // integer part
        float t = (phase & 0xFFFF) * (1.0f / 65536.0f);

        int n = (int) idx - 1;

        // xm1 and x2 are only used for derivative estimation
        // we interpolate between x0 and x1
        float xm1 = buffer[n];
        float x0 = buffer[n + 1];
        float x1 = buffer[n + 2];
        float x2 = buffer[n + 3];

        // estimate derivatives by finite differences
        float m0 = 0.5f * (x1 - xm1); // derivative at x0
        float m1 = 0.5f * (x2 - x0);  // derivative at x1

        // calculate coefficients
        // Cubic Hermite interpolation (Catmull-Rom spline with Tension = 0)
        float c = m0;
        float v = x0 - x1;
        float w = c + v;
        float a = w + v + m1;
        float b = w + a;
        float d = x0;

        // cubic polynomial in Horner form.
        // actually a*t^3 + b*t^2 + c*t + d
        return (((a * t - b) * t + c) * t + d);
}
```

Listing 2: Hermite interpolation based on Laurent de Soras' implementation from `https://www.musicdsp.org/en/latest/Other/93-hermite-interpollation.html`

Although the function reads four samples $y[n-1], y[n], y[n+1], y[n+2]$, the actual Hermite interpolation is performed only between the two central points, $y[n] =$ `x0` and $y[n+1] =$ `x1` (lines 10–11). The two outer points, $y[n-1] =$ `xm1` and $y[n+2] =$ `x2`, are used exclusively to estimate the derivatives at the interpolation points (lines 9, 12).
Since the derivatives of a discrete-time audio signal are not directly available, they are approximated using finite differences (lines 15–16):

$$m_0 = \frac{x_1 - x_{-1}}{2}, \quad m_1 = \frac{x_2 - x_0}{2}$$

This ensures a smooth cubic curve between x_0 and x_1 while only requiring the immediate neighbours of the interpolated points.
So even though the actual interpolation is done between points `x0` and `x1` of the function, the preceding sample $n-1$, as well as the following neighbour $n+2$ are also required as inputs.
This approach on the Hermite interpolation, where tangents are estimated from neighbouring samples, is commonly referred to as *Catmull-Rom spline interpolation.* [4, p.11]
The coefficients a, b, c, d are calculated from the sample values and the estimated derivatives to satisfy both the interpolation points and the slope constraints at those points (lines 20–25).
The algorithm returns the result by computing a cubic polynomial using the nested Horner form (line 29) [2]:

$$H(t) = (((a \cdot t - b) \cdot t + c) \cdot t + d), \quad t \in [0, 1]$$

which is algebraically equivalent to the standard form

$$H(t) = at^3 + bt^2 + ct + d, \quad t \in [0, 1]$$

but requires fewer arithmetic operations and is therefore preferred in DSP and CPU implementations.

4.6 Evaluation

4.6.1 Measurement Setup

To evaluate interpolation quality, a 100 Hz sawtooth wave was loaded into the tape buffer. A sawtooth was chosen as the test signal because it contains both even and odd harmonics, maximising the number of spectral components visible in the FFT output.
The two interpolation methods under comparison were ZOH and the 4-point cubic Hermite with Catmull-Rom tangents. Each method was recorded separately at the following playback speeds: 0.25× (−2 octaves), 0.33×, 0.5× (−1 octave), 0.75×, 1.0× (unity), 1.5×, 2.0× (+1 octave), and 4.0× (+2 octaves).
To eliminate possible parameter jitter and value inaccuracy from the ADC, the playback speed was fixed at compile time via a macro in `project_config.h`, bypassing the control interface entirely:

```
1          #define CONFIG_TAPE_PITCH_OVERRIDE 4.0f
```

The output was captured at $f_s = 48$ kHz as 24-bit `.wav` files using a professional RME Fireface UFX II audio interface.

4.6.2 Results

Figure 5 shows the FFT magnitude spectra of the recorded signals for each playback speed.

../images/interpolation/fig_interpolation_fft.pdf

Figure 5: FFT of 100Hz Saw signal - interpolated via Hermite and ZOH with different playback speeds

At unity playback (1.0×), both methods produce nearly identical spectra: the harmonic series of the sawtooth is clearly visible with no significant aliasing, as no interpolation is required when the playhead advances at exactly one sample per output step.

As the playback speed decreases below 1.0×, the effective upsampling ratio increases and the interpolator is called more frequently, making its quality increasingly audible. At 0.5× and below, ZOH shows clearly visible aliasing humps between the signal harmonics — folded spectral energy from above Nyquist. At 0.25×, a second aliasing hump appears, consistent with the broad sidelobe structure predicted by the ZOH frequency response in Section 4.4.

The Hermite interpolator shows substantially lower aliasing across all upsampling ratios. Even at 0.25×, where ZOH produces prominent aliasing humps, the Hermite output retains only a small residual artefact — consistent with the suppressed sidelobe structure of the Catmull-Rom kernel discussed in Section 4.4.

Playback Speed	**ZOH**	**Hermite (Catmull-Rom)**
4.0× (+2 oct)	Clean	Clean
2.0× (+1 oct)	Clean	Clean
1.5×	Clean	Clean
1.0× (unity)	Clean	Clean
0.75×	Minor aliasing	Clean
0.5× (−1 oct)	Visible aliasing humps	Minor aliasing
0.33×	Strong aliasing	Minor aliasing
0.25× (−2 oct)	Strong aliasing, double hump	Residual artefact only

Table 1: Qualitative comparison of aliasing artefacts at each playback speed.

5 Focus Point 2: Reverb

A digital reverb effect was chosen as a natural complement to the pitch shifting functionality of the module. Reverberation widens the sonic palette of the sampler significantly: by pushing parameters to extremes, the effect can produce sounds ranging from harsh metallic resonances to wide, lush, near-infinite spatial decay – far beyond conventional room simulation.

5.1 Problem Statement

The central challenge is implementing a perceptually plausible, *flexible* reverb on microcontroller-class hardware, where computational resources and memory are strictly limited. A reverb algorithm is required that is both computationally lightweight and perceptually convincing, while exposing a set of parameters for real-time RT60 control, room size and wet mix amount.
Room simulation can become very complex when attempting to physically model the acoustic behaviour of a space. Important aspects include correct physical behaviour of sound waves, perceptually convincing reverberation, and early and late reflection simulation. **juliuso.smithPhysicalSignalProcessing**
All of these are relevant considerations, but implementing each in depth is out of scope for this project, as the reverb is merely the DSP component of a larger system. The goal is a straightforward implementation that demonstrates the real-time capabilities of the hardware while producing aesthetically interesting spatial effects. For this reason, one of the earliest concepts of artificial reverberation based on discrete-time signal processing was chosen – constructed by Schröder in the early 1960s and known as the *Schröder Reverberator*.

5.2 Theory of the Schröder Reverb

Schröders original proposal is a combination of *comb* and *allpass* filters. **m.r.schroederJournalAudioEngineering1962** 223] The implementation structure used in this project includes a bank of parallel comb filters, combining into a series of allpass filters. The variant used here – including lowpass-feedback comb filters – is also known as the **Moore** variant of the reverberator.
Figure 6 shows the grobe structure of the implemented reverb. Cursive parameters, such as filter coefficient *alpha*, the *feedback* gain and *delay length*, are implemented as controllable by outside of the component.
The combination of parallel comb filters feeding into series allpass filters is a psychoacoustically motivated design. The comb filters produce the characteristic frequency-domain coloration and exponential decay of a reverberant space, but their echo density is too sparse on its own. The allpass filters — also called *diffusers* — increase the echo density without adding frequency coloration, since their magnitude response is flat across all frequencies. Together, the structure achieves both a perceptually convincing decay envelope and a sufficiently dense late reverberation tail. **juliuso.smithPhysicalSignalProcessing**

5.2.1 Feedback Comb Filter

At the basis of the *Schröder Reverberator* is a structure of parallel comb filters. Comb filters are fundamental building blocks for digital audio effects. Its most obvious example is a simple echo implementation **juliuso.smithPhysicalSignalProcessing**.
The most simple variant is a *feedforward* comb filter. It takes the input signal $x[n]$ and adds a delayed copy, shifted by D samples, to produce the output $y[n]$. Since the delayed signal is not fed back into the input, each reflection occurs only once.
In theory, a room reverberation simulation can be constructed using only *feedforward* comb filters, where each reflection is modelled by a separate filter instance. In practice however, this becomes computationally expensive for a convincing reverb with many reflections. **juliuso.smithPhysicalSignalProcessing**

../images/schroeder_reverb/schroeder_reverb.drawio.pdf

Figure 6: Schroeder Reverb Block Diagram

../images/schroeder_reverb/comb.drawio.pdf

Figure 7: Feedback Comb Filter

Where the *feedforward* comb filter implements an FIR filter, the *feedback* comb filter (Figure 7) takes the delayed output and feeds it back to the input. This filter type is also known as an IIR (Infinite Impulse Response) or *recursive* filter, and is described by the difference equation:

$$y[n] = x[n] + \alpha \cdot y[n - D]$$

where α is the feedback gain and D the delay in samples.
The feedback comb filter can be understood as a physical model of a series of exponentially decaying echoes, resembling a signal reflecting between two parallel walls. For this reason, feedback comb filters are preferred in reverb design, offering a more natural decay while remaining computationally efficient.**juliuso.smithPhysicalSignalProcessing**
The feedback coefficient α must be less than 1 in magnitude for the filter to be stable. Otherwise each echo would be louder than the previous one, causing the output to diverge and become unstable.
A feedback gain close to 1 produces a very slowly decaying tail, which can be an interesting artistic effect but must be used with caution.
In Figure 8 the decaying echoes at uniform intervals of D samples are clearly visible in the impulse response. The characteristic frequency response resembling a comb gives the filter its name. The evenly spaced peaks arise because constructive interference occurs at every integer multiple of f_s/D, producing resonances separated by $1/D$ in normalized frequency.
Essentially the peaks are frequencies where the delay D is an exact multiple of the period of that frequency. The delayed copy arrives back perfectly in phase with the new input, and they reinforce each other. At the troughs, the input signal is exactly *out of phase* with the fed-back delayed signal, causing cancellation.

../images/schroeder_reverb/fig_comb_filter_response.pdf

Figure 8: Impulse and Frequency Response of Feedback Comb Filter with different feedback gain α

Lowpass-Feedback Comb Filter Instead of using a standard feedback comb filter, this project employs a *lowpass-feedback* comb filter (LBCF), which places a filter $H_f(z)$ in the feedback path alongside the feedback gain α.

../images/schroeder_reverb/lp_filtered_comb.drawio.pdf

Figure 9: Filtered-Feedback Comb Filter with 1st order Lowpass

The transfer function in the z-domain is given by:

$$H_{LBCF}(z) = \frac{z^{-D}}{1 - \alpha \cdot H_f(z) \cdot z^{-D}}$$

Note that the transfer function is given here with the output taken after the delay, differing slightly from the notation in **juliuso.smithPhysicalSignalProcessing**.
Using a first-order lowpass $H_f(z) = H_{lp}(z)$ with a damping coefficient d:

$$H_{lp}(z) = \frac{1 - d}{1 - d \cdot z^{-1}}$$

the full transfer function becomes:

$$H_{LBCF}(z) = \frac{z^{-D}}{1 - \alpha \frac{1-d}{1-d \cdot z^{-1}} z^{-D}}$$

The effect of the lowpass in the feedback path causes high frequencies to decay faster than low frequencies, producing a more natural reverb tail. Adding a low-pass filter in the feedback loop can effectively recreate the losses that happen when a signal travels back and forth between two places, as happens in real rooms. **juliuso.smithPhysicalSignalProcessing**

Impulse and Frequency Response In Figure 10, the effect of the lowpass filter in the feedback path is clearly visible.

../images/schroeder_reverb/fig_comb_filter_lp_feedback_r

Compared to the pure feedback comb filter in Figure 8, the impulse response shows non-zero samples between the echo peaks, caused by the smearing effect of the one-pole IIR filter.
In the frequency response, the comb peaks are attenuated at higher frequencies, as the lowpass reduces the feedback gain progressively with frequency — resulting in shorter decay times for high frequencies than for low frequencies.

5.2.2 Allpass Filter

The allpass filter passes all frequencies at equal gain. Unlike lowpass or highpass filters, it does not attenuate any frequency band. Instead, it affects only the *phase* of the signal, introducing frequency-dependent delays while leaving the magnitude response flat at unity gain across all frequencies. In the context of the Schröder reverberator, allpass filters are used as *diffusers*: they increase the echo density of the late reverberation without coloring the sound, complementing the tonal decay provided by the comb filter bank. **juliuso.smithPhysicalSignalProcessing**

../images/schroeder_reverb/allpass.drawio.pdf

Figure 11: Allpass Filter

The allpass filter can be constructed by cascading a *feedforward* and *feedback* comb filter delay line (see Section 5.2.1). Figure 11 illustrates the combination of the two comb filter variants. **juliuso.smithPhysicalSignalProces**

When inspecting the transfer function:

$$H(z) = \frac{\alpha + z^{-D}}{1 + \alpha z^{-D}}$$

It becomes clear, how the Schröder allpass works. The numerator and denominator of $H(z)$ are mirror images of each other.

Whatever the feedback amplifies at a certain frequency, the feedforward cancels exactly, leaving only a phase shift (the delay in time-domain) behind. This causes the totally flat frequency response of the *allpass* filter. Note the similar impulse response to the feedback comb filter 8

../images/schroeder_reverb/fig_allpass_response.pdf

5.3 Design and Implementation

The implemented reverberator follows a simplified variant of the *Freeverb* **freeverb-github** variant of the Schröder structure – a public-domain C++ program, used extensively in the free-software community – adapted here for embedded C on a microcontroller target.
It consists of four parallel lowpass-feedback comb filters summing into two series allpass filters, per channel **juliuso.smithPhysicalSignalProcessing**. The full z-domain block diagram is shown in Figure 13.

../images/schroeder_reverb/schroeder_reverb_block_detailed.drawio.pdf

Figure 13: Schroeder Reverb Block Diagram

Delay Line Structure Each filter — comb or allpass — is backed by a statically allocated circular float buffer that holds the sample history required by the delay line. Both filters share the same `sr_delay_t` struct, which stores the buffer pointer, its maximum allocated `size`, the currently active `length`, and a write index `idx`. Separating `size` from `length` allows the active delay time to be changed at runtime without reallocating memory. The struct also carries filter-specific state: `feedback` gain and `lp_state` for the comb filter's damped feedback path.

```c
typedef struct {
        float feedback;
        float* buf;
        uint32_t size;   /**< Buffer capacity in samples. */
        uint32_t idx;
        uint16_t length; /**< Active delay length in samples (<= size). */

        float lp_state;
        float lp_alpha;  /**< One-pole LP coefficient in the comb feedback path.
        ↪  */
} sr_delay_t;
```

Listing 3: delay-line struct type definition

The functionality of both, the comb- and allpass filters are implemented in the respective `*_process` functions:

```c
1  static float comb_process(sr_delay_t* d, float in) {
2          // calculate read index with wraparound depending on set .length parameter
3          uint32_t read_idx = (d->idx + d->size - d->length) % d->size;
4          float y = d->buf[read_idx];
5
6          float feedback_signal = y * d->feedback;
7
8          // one-pole lowpass on each combs fb path.
9          d->lp_state = (1.0f - d->lp_alpha) * feedback_signal + d->lp_alpha *
           ↪  d->lp_state;
10         d->buf[d->idx] = in + d->lp_state;
11
12         d->idx++;
13         if (d->idx >= d->size)
14         d->idx = 0;
15
16         return y;
17 }
18
19 static float allpass_process(sr_delay_t* d, float in) {
20         // calculate read index with wraparound depending on set .length parameter
21         uint32_t read_idx = (d->idx + d->size - d->length) % d->size;
22         float buf = d->buf[read_idx];
23
24         float y = -in + buf;
25         d->buf[d->idx] = in + buf * d->feedback;
26
27         d->idx++;
28         if (d->idx >= d->size)
29         d->idx = 0;
30
31         return y;
32 }
33
```

Listing 4: process functions of comb and allpass filters

Both underlying `sr_delay_t` data structures advance with a write index `idx` that advances by one sample each tick and wraps modulo size. The delay tap is read from position `(idx + size - length) % size` (line 3 for comb, 21 for allpass), placing it exactly `length` samples behind the write head. This avoids any branch on the read side and keeps both operations O(1).

For the comb filter, the fed-back sample passes through a one-pole lowpass (line 9) before being added to the input and written into the buffer (line 10), implementing the damped feedback loop in-place.
The allpass filter uses the same ring structure but applies the Schroeder allpass topology directly (lines 22–23): the delayed sample `buf` is summed with the negated input for the output, while the write combines the input with the unfiltered delayed sample scaled by `feedback`, preserving the all-pass magnitude response while dispersing phase.

Static Memory Allocation Twelve buffers are allocated in total: eight comb buffers of `MAX_COMB_LEN` = 1620 samples and four allpass buffers of `MAX_AP_LEN` = 600 samples. At 32-bit float precision (4 bytes per sample), the total static memory footprint is:

$$(8 \times 1620 + 4 \times 600) \times 4 \text{ bytes} = 61.4 \text{ kB}$$

The buffer size and thus the maximum delay lengths are directly correlated to the maximum room size of the algorithm. Increasing the base delay lengths would produce a larger perceived space at `size = 1`, at the cost of higher memory consumption. The current values were chosen to provide an aesthetically pleasing maximum room size while maintaining a conservative memory footprint, preserving available RAM for the sample playback buffers in the system.

Delay Length Selection Using mutually prime delay line lengths — values sharing no common factors — helps avoid spectral clustering and unnatural resonances in the frequency response. When delay lengths share common factors, their resonant frequencies align, causing certain frequencies to be disproportionately reinforced. By choosing mutually prime lengths, the resonances are maximally spread across the spectrum, resulting in a evenly decay envelope. The delay lengths were selected empirically by ear from prime numbers, starting from a base set and scaled by a room size parameter. The chosen base delay lengths are:

	Comb Filter Delays (samples)			
	1	2	3	4
Left	1553	1613	1499	1427
Right	1567	1601	1487	1433

	Allpass Delays (samples)	
	1	2
Left	223	557
Right	229	563

The slight asymmetry between left and right channels provides stereo decorrelation, widening the perceived sound field without additional processing.

Stability and Feedback Scaling To prevent instability as the room size parameter is reduced, the feedback gain is scaled automatically using two macros:

```
#define COMB_FEEDBACK(size)   (0.97f + 0.01f * (1.0f - size))
#define ALLPASS_FEEDBACK(size)(0.68f + 0.01f * (1.0f - size))
```

As the room size decreases, delay lines shorten and the filter accumulates energy more rapidly. The macros compensate by slightly increasing the base feedback value as size decreases, maintaining a perceptually consistent decay across the full parameter range. The feedback value passed in by the user is additionally clamped to $[0, 0.999]$ before being applied, ensuring the filter never reaches the instability threshold of $\alpha = 1$. A minimum room size of 0.30 is enforced to prevent the delay lines from becoming so short that the filter becomes difficult to stabilise.

Controllable Parameters The reverb exposes four parameters via a public API:

`wet` Controls the dry/wet balance, with `dry` $= 1 -$ `wet` enforced automatically.

`feedback` (α) Scales the RT60 decay time, routed through the stability macros to compensate for room size.

`size` Uniformly scales all delay lengths while preserving their prime relationships.

`lp_alpha` (d) Controls the cutoff of the one-pole IIR filter in each comb feedback path, attenuating high frequencies and producing a more natural, air-absorbed decay.

Parameter Coupling The feedback gain and room size parameters are implicitly coupled through the stability macros. The `size` scalar is stored in the `schroeder_stereo_t` struct so that `schroeder_rev_set_feedback` can reference it when computing the scaled feedback values:

```
float comb_fb = feedback * COMB_FEEDBACK(rev->size);
```

This means that calling public API function `schroeder_rev_set_size` alone changes the effective feedback range, even without a subsequent call to `schroeder_rev_set_feedback`.

5.4 Evaluation

5.4.1 Impulse Response Decay

Figure 14 shows the measured envelope decay of the reverb unit under varying feedback and room size parameters. Each curve represents the smoothed amplitude envelope of a 20 ms white noise burst passed through the reverb and plotted until the signal reaches −60 dB.

../images/schroeder_reverb/fig_reverb_envelope.pdf

Figure 14: Reverb Decay Times of a 20 ms White Noise Burst

Feedback and Room Size As expected, higher feedback coefficients result in longer decay times. At maximum feedback ($fb = 0.975$), the reverb tail extends significantly longer than at minimum feedback ($fb = 0.442$), which decays rapidly toward the noise floor. The most important validation is the stability at maximum feedback, on both maximum and minimum room sizes.
The room size parameter scales the delay line lengths of the comb filters. At maximum room size, the decay is smoother and more diffuse, while at minimum room size the shorter delay lines produce a denser, more colored decay.

The parameter boundaries were determined empirically during testing, chosen to represent the range that produces the most musically useful results — long, dense decay at the upper end, and short, subtle ambience at the lower end — while ensuring the filters remain stable throughout.

RT60 The −60 dB decay time (RT60) can be read directly from the x-axis. At maximum feedback and room size, the RT60 is approximately 5 s, while at minimum feedback it falls below 0.5 s. Notably, at minimum room size the RT60 reaches only approximately 3.25 s even at maximum feedback — significantly shorter than the 5 s observed at maximum room size. During testing, it became clear that feedback scaling (described in Section 5.3) alone cannot fully compensate for the shorter decay at smaller room sizes — increasing feedback further risks instability before the RT60 of larger room sizes can be matched.

5.4.2 Spectrogram

Figure 15 shows the spectrogram of the reverb tail at maximum feedback ($fb = 0.975$, size $= 1.0$), comparing the comb filter lowpass enabled and disabled. The input is a 20 ms white noise burst, plotted over 5 s of decay.

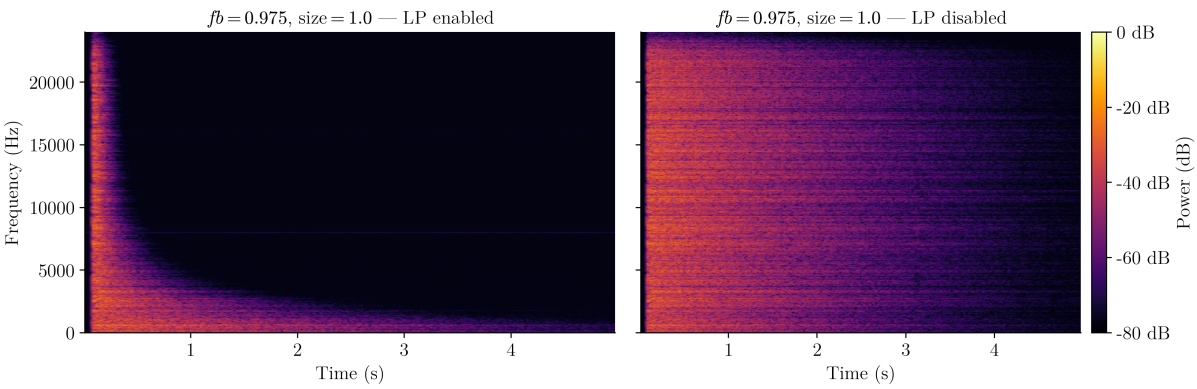


Figure 15: Reverb Tail Spectrogram at comb lowpass filter on feedback settings

LP enabled With the lowpass active ($d = 0.4$), high-frequency energy decays significantly faster than low-frequency content. The rolloff is clearly visible as the bright energy concentrates toward the lower frequencies over time, producing the characteristic warm, darkening tail associated with natural room acoustics.

LP disabled Without lowpass damping, energy is distributed more evenly across the frequency spectrum throughout the decay, resulting in a brighter and more unnatural sounding tail.

Summary Both variants have usable applications in music production. Engaging the lowpass produces a more natural sounding reverb tail, modelling the frequency-dependent absorption of a natural acoustic space, where high frequencies are absorbed more rapidly than low frequencies.

An interesting observation is that the frequency content of the reverb tail extends almost to the Nyquist limit of $f_s/2$ (24 kHz), confirming that neither the used audio interface (RME Fireface UFX II interface **FirefaceUFXII**) nor the embedded system introduce any significant bandlimiting, and the observed high-frequency content reflects the true output of the reverb algorithm.

6 Verification and Evaluation

6.1 Hardware Testing

6.1.1 Power and safety tests

6.1.2 Signal integrity measurements

6.2 ADC Input Range and Signal Levels

The TLV320AIC audio codec used in this project is powered with 1.8 V ($AVDD$), resulting in a common-mode voltage of

$$V_{\mathrm{CM}} = \frac{AVDD}{2} = 0.9\,\mathrm{V}.$$

According to the ADC electrical characteristics in the TLV320AIC datasheet [8, p. 9], the ADC is configured for a full-scale input of $\pm 0.53\,\mathrm{V}$ around V_{CM}. This corresponds to a maximum peak-to-peak voltage of

$$V_{\mathrm{pp,max}} = (V_{\mathrm{CM}} + 0.53\,\mathrm{V}) - (V_{\mathrm{CM}} - 0.53\,\mathrm{V}) = 1.06\,\mathrm{V_{pp}}.$$

Any input signal exceeding this range results in saturation of the ADC, producing clipped samples. In our measurements, the input sine wave reached a maximum of 1.4 V and a minimum of 0.08 V, which is outside the ADC's safe range. Consequently, the recorded sample buffers contain values at the int16 limits (-32768 to 32767).
The input stage includes a voltage divider, originally designed with resistor values of $1.8\,\mathrm{k\Omega}/12\,\mathrm{k\Omega}$, resulting in an attenuation of 0.15. With a typical Eurorack input signal of $10\,\mathrm{V_{pp}}$, this configuration drives the TLV320AIC ADC beyond its maximum input range.

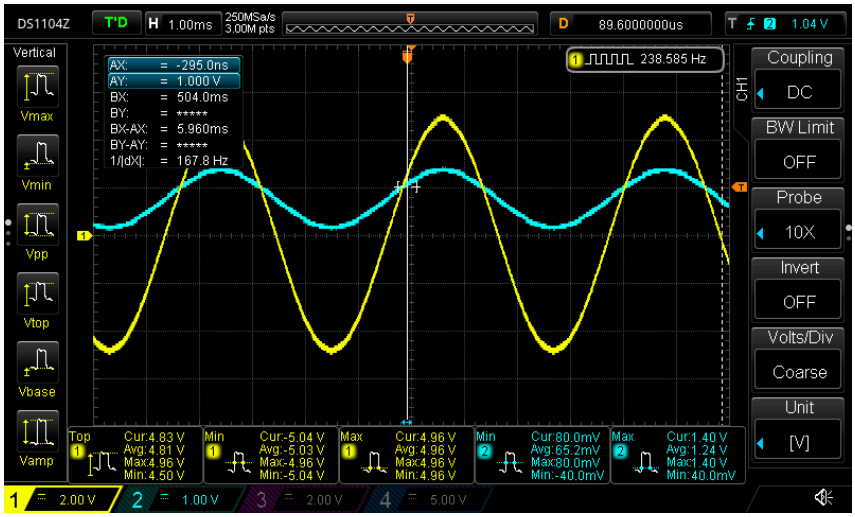


Figure 16: Attenuation of a $10\,\mathrm{V_{pp}}$ Eurorack-level signal to a biased $1.32\,\mathrm{V_{pp}}$ ADC input.

To provide additional headroom and prevent clipping, the voltage divider was adjusted to $1\,\mathrm{k\Omega}/12\,\mathrm{k\Omega}$, yielding an attenuation of approximately 0.084. This reduces the $10\,\mathrm{V_{pp}}$ input signal amplitude to about $0.77\,\mathrm{V_{pp}}$, which is safely within the ADC's input range.
This configuration also allows more headroom for larger signals. For lower-amplitude inputs, the TLV320AIC's programmable gain amplifier (PGA) can be used to boost the signal, thereby utilizing the full resolution of the ADC.

6.2.1 Noise and distortion measurements

6.3 Firmware Testing

6.3.1 Driver verification

6.3.2 Timing and latency analysis

6.3.3 Calibration accuracy

6.4 System-Level Validation

End-to-end signal tests/loopback?
Eurorack compatibility tests

6.5 Summary

6.5.1 Summary

Discussion of Limitations

6.5.2 Ausblick

References

[1] A. Baikady and B. V. Uma, "Area and Power Efficient Architecture for Direct Digital Frequency Synthesizer," in *2021 10th International Conference on Internet of Everything, Microwave Engineering, Communication and Networks (IEMECON)*, Dec. 2021, pp. 01–07. DOI: 10.1109/IEMECON53809.2021.9689136 Accessed: Jan. 19, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/9689136/

[2] C. S. Burrus, J. W. Fox, G. A. Sitton, and S. Treitel, "Horner's Method for Evaluating and Deflating Polynomials,"

[3] "How to select a compatible crystal and load capacitors for STM32 with layout guidelines," Accessed: Dec. 29, 2025. [Online]. Available: https://community.st.com/t5/stm32-mcus/how-to-select-a-compatible-crystal-and-load-capacitors-for-stm32/ta-p/780236

[4] Olli Niemitalo, "Polynomial Interpolators for High-Quality Resampling of Oversampled Audio," Sep. 2001. Accessed: Jan. 18, 2026. [Online]. Available: https://yehar.com/blog/wp-content/uploads/2009/08/deip.pdf

[5] Smith, Julius O. "Digital Audio Resampling Home Page," Accessed: Jan. 16, 2026. [Online]. Available: https://ccrma.stanford.edu/~jos/resample/resample.pdf

[6] STMicroelectronics, *Stm32h7a3ri Datasheet*, STMicroelectronics.

[7] Texas Instruments, *General ADC Calibration*, Texas Instruments. Accessed: Jan. 31, 2026. [Online]. Available: https://e2e.ti.com/cfs-file/__key/CommunityServer-Discussions-Components-Files/156/3364.General-ADC-calibration.pdf

[8] Texas Instruments, *Tlv320aic3204 Datasheet*, Texas Instruments. Accessed: Aug. 12, 2024. [Online]. Available: https://www.ti.com/lit/ds/symlink/tlv320aic3204.pdf?ts=1720871960400